Seniority Portal - Enterprise Documentation Package

Generated on: 2026-07-26

1. Executive Overview

Application Name

The application is documented in the repository as Seniority Portal. The deployed browser UI brands the system as CSI Thoothukudi-Nazareth Diocese - Employment Priority List.

Business Purpose

Seniority Portal is a frontend web application that publishes and manages employment priority lists for the CSI Thoothukudi-Nazareth Diocese. It helps authorized stakeholders and applicants view candidate priority rankings for school employment and clergy ordination categories, filter lists by ministry or education attributes, and export formal reports as PDFs.

Problems Solved

- Converts spreadsheet-based candidate records into searchable, ranked priority lists.
- Applies repeatable ranking rules instead of requiring manual re-sorting.
- Separates active candidates from appointed, held, and exited candidates.
- Provides PDF exports for priority lists, appointment reports, and exit registers.
- Displays ChangeLog audit rows with official document preview and download actions.
- Supports English and Tamil UI text for operational accessibility.
- Provides an implemented vacancy dashboard and map for school application workflows, although the route is disabled by feature flag in the current configuration.

Intended Users

- Diocese administrative staff who maintain appointment and priority records.
- Pastorate and council stakeholders reviewing candidate lists.
- Candidates or applicants checking priority list visibility.
- School appointment workflow teams using appointment, hold, and exit reports.

Key Benefits

- Browser-first deployment: no application server is required for the main runtime.
- Live data loading: published Google Sheets CSV/JSON can be fetched directly by the browser.
- Transparent business rules: ranking logic is encoded in src/app/config/seniorityRules.ts.
- Export-ready reporting: client-side PDF generation is implemented with jspdf and jspdf-autotable.
- Operational controls: feature flags enable or suppress sections, downloads, IDs, address fields, and routing.

2. Application Complexity Assessment

Technical Complexity

This is a single-page React application built with Vite, but it contains significant business and data complexity. The application maps multiple source data schemas into normalized candidate models, applies category-specific ranking logic, refreshes remote data periodically, handles appointment state, and supports report generation.

The codebase includes:

- Main React app under src/app.
- Shared theme package under packages/ui-theme.
- Reusable starter template under templates/csi-starter.

- Public static assets and generated data cache under public.
- Node-based Google Sheet sync script at sync-google-sheet-to-json.js.
- Docker and Vercel deployment configuration.

Development Effort Required

The implementation represents more than a static table. It required:

- CSV and JSON ingestion logic.
- Date, year, TET, appointment, hold, and exit normalization.
- Ranking algorithms for three business domains.
- Dynamic filtering, searching, pagination, and sort-mode switching.
- Responsive tables with conditional columns.
- PDF report layout, footer, watermark, and export behavior.
- Chart and map-based visualizations.
- Theme and language persistence.
- Deployment configuration for static hosting and containerized hosting.

Architectural Challenges

- Remote spreadsheet data can contain inconsistent column names and spelling variations.
- Published Google Sheets can return HTML error pages instead of CSV, so the app checks response content.
- Ranking rules differ by school type and view mode.
- Appointment rows are ranked separately from seniority rows.
- UI must remain useful across wide tables and mobile viewports.
- The main application is frontend-only, so all public data URLs and source data are accessible to clients.

Integration Complexity

The application integrates with:

- Published Google Sheets CSV endpoints.
- Optional JSON cache file at public/seniority-data.json.
- Browser storage through localStorage.
- Browser Fetch API.
- OpenStreetMap tiles through Leaflet.
- Google Maps direction links generated from coordinates or map links.
- Static PDF form files under public/forms.

User Experience Considerations

The application includes:

- Sticky header with logo, theme toggle, and language toggle.
- Splash screen using public/diocese-logo.png.
- Priority type selector for High/Higher Secondary, Elementary/Middle, and Clergy.
- Compact filter bar.
- Search across candidate identity, geography, qualification, category, and status fields.
- Visual dashboard modal using charts.
- Appointment, exit register, and ChangeLog views.
- Pagination at 20 rows per page.

Security Considerations

The application is frontend-only. It does not implement authentication, authorization, server-side access checks, or private data protection in this repository. Any published CSV/JSON URL or Drive PDF URL used by the browser should be treated as public to technical users because browser DevTools and network traffic can reveal client-fetched URLs and response data. Feature flags can hide fields such as member ID, address, pincode, and raw

Drive links from the UI/PDF, but they do not secure the underlying data source if the browser can fetch it.

The ChangeLog UI intentionally shows only View and Download document actions and does not show an Open in Drive action. This improves presentation and reduces accidental sharing, but it does not make a public Drive file or published CSV private. True URL secrecy requires a backend/serverless API with authentication, role checks, private Google credentials, and server-side row/document proxying.

Scalability Considerations

The current bundled JSON cache contains 3,048 high school records, 8 elementary records, and 11 vacancy records. The app performs filtering, ranking, searching, and PDF generation in the browser. This is appropriate for thousands of rows, but substantially larger datasets may require virtualization, server-side filtering, or API-backed pagination.

3. System Architecture

Overall Architecture

The discovered architecture is a static frontend single-page application.

```
User Browser
  |
  | loads static HTML/CSS/JS from hosting
  v
React + Vite SPA
  |
  | fetches CSV/JSON with cache-busting
  v
Published Google Sheets / public JSON cache
  |
  | normalized in browser
  v
Ranking, filtering, searching, reports, charts, map
```

No traditional backend API or database implementation exists in the workspace.

Frontend Architecture

Primary frontend files:

- src/main.tsx: React entrypoint.
- src/app/App.tsx: theme initialization, language provider, router provider, toaster.
- src/app/routes.tsx: browser routes.
- src/app/pages/Root.tsx: shared layout.
- src/app/pages/Dashboard.tsx: main priority list workflow.
- src/app/pages/ApplyPage.tsx: application forms, vacancy dashboard, and map workflow.
- src/app/components/*: tables, filters, charts, header, footer, guidelines, map.
- src/app/utils/*: data fetch, search helper, PDF utilities.

Backend Architecture

No running backend server is implemented for the application runtime. The closest supporting service is sync-google-sheet-to-json.js, a Node script that can fetch published Google Sheet CSV data and write a static JSON cache file to public/seniority-data.json.

Service Architecture

Runtime services are external or browser-native:

- Static hosting service, such as Vercel, Netlify, Cloudflare Pages, Nginx, or another static host.
- Published Google Sheets endpoints as CSV data providers.

- Browser storage for language and theme preferences.
- OpenStreetMap tile service for vacancy mapping.

Component Relationships

```
App
  -> LanguageProvider
  -> RouterProvider
       -> Root
            -> Header
            -> Dashboard
                 -> FilterSidebar
                 -> SearchBar
                 -> SeniorityTable
                 -> DashboardVisual
                 -> pdfUtils dynamic imports
            -> ApplyPage (disabled by current flag)
                 -> SchoolVacancyMap
                 -> Chart.js charts
            -> AppGuidelines
            -> SiteFooter
```

Communication Methods

- Browser fetch() retrieves CSV or JSON.
- React state and memoized selectors compute rankings and filtered views.
- URL search params record appointment view state.
- localStorage stores theme and language preferences.
- Client-side downloads are triggered with jsPDF or temporary anchor elements.

Data Flow

```
Google Sheets CSV endpoints
  -> fetchGoogleSheetData()
  -> parseCSV() / normalizePayload()
  -> Dashboard mapping functions
  -> candidate arrays by domain
  -> filters + search + ranking
  -> SeniorityTable / DashboardVisual / PDF exports
```

Optional local sync flow:

```
sync-google-sheet-to-json.js
  -> HTTPS fetch Google Sheet CSV
  -> parse CSV
  -> hash payload
  -> write public/seniority-data.json when changed
```

4. Technology Stack Analysis

Languages

- TypeScript and TSX: application source and React components.
- JavaScript: Google Sheet sync script.
- CSS: application styling and shared theme package.
- JSON: package, config, data cache, and deployment metadata.
- CSV: source/fixture candidate data.

Frameworks and Libraries

- React 18.3.1: component model and application state.
- React DOM 18.3.1: browser rendering.

- React Router 7.18.1: SPA routing.
- Vite 6.4.3: development server and build system.
- Tailwind CSS 4.1.12 with @tailwindcss/vite: utility styling pipeline.
- Radix UI primitives: checkbox, dialog, label, scroll area, slot.
- lucide-react: icons.
- date-fns: date formatting and age calculation.
- Chart.js and react-chartjs-2: dashboards and vacancy charts.
- Leaflet and react-leaflet: school vacancy map.
- Framer Motion: dashboard modal animation.
- jsPDF and jspdf-autotable: PDF report generation.
- Sonner: toast infrastructure.
- dotenv: environment loading for the Node sync script.
- class-variance-authority, clsx, tailwind-merge: UI class composition.
- next-themes is listed as a dependency but the implemented theme manager is local in packages/ui-theme.

Development and Build Tools

- npm scripts currently present: dev, build, preview.
- vite.config.ts configures React, Tailwind, alias @ to src, and raw asset support for SVG/CSV.
- Dockerfile builds with Node 20 Alpine and serves dist with Nginx 1.27 Alpine.
- vercel.json rewrites all routes to index.html for SPA fallback.

Deployment Technologies

- Static build output: dist.
- Vercel SPA fallback is configured.
- Docker runtime is configured with Nginx.

5. Data Management Architecture

Discovered Data Sources

The application uses published Google Sheets CSV endpoints and optional static JSON.

Default endpoints in src/app/utils/fetchGoogleSheetData.ts:

- High school: published Google Sheet, gid=0.
- Elementary school: published Google Sheet, gid=882704265.
- Clergy ordination: published Google Sheet, gid=271291357.
- ChangeLog: published Google Sheet, gid=246990650.

Vacancy endpoint in src/app/pages/ApplyPage.tsx:

- School vacancy sheet, gid=1387124453.

Static cache:

- public/seniority-data.json.

Local CSV resources:

- data/high-higher-secondary-converted.csv.
- elementary-mock-data.csv.

Static form resources:

- public/forms/high-higher-secondary-school-form.pdf.
- public/forms/elementry-middle-school-form.pdf.

Data Counts Observed

public/seniority-data.json contains:

- highSchool: 3,048 rows.

- elementarySchool: 8 rows.
- schoolVacancies: 11 rows.
- syncedAt: 2026-03-08T17:03:04.870Z.
- sources: published Google Sheet source URLs.
- changeLog: optional audit rows when synced.

data/high-higher-secondary-converted.csv contains 3,048 data rows.

elementary-mock-data.csv contains 8 data rows.

Storage Methods

No database schema, database client, migrations, or server persistence layer exists in the workspace. Data is stored externally in published sheets, optionally mirrored into a static JSON file, and processed in browser memory after load.

Browser storage is used for preferences:

- theme-mode in localStorage.
- seniority_language in localStorage.

Data Synchronization

The browser fetch path refreshes data every 60 seconds in Dashboard.tsx. It retries failed loads and only updates React state when a stable JSON hash of normalized records changes.

The Node sync script supports one-shot and watch mode behavior in code, but package.json does not currently define sync:sheet or sync:sheet:watch scripts even though README.md mentions them.

Data Validation and Normalization

The application performs defensive normalization:

- CSV parser handles quoted values and BOM headers.
- Fetch layer detects HTML responses when CSV/JSON is expected.
- Date parser supports dd.mm.yy, dd.mm.yyyy, slash and dash variants, and native date parsing fallback.
- Column access uses loose matching for spelling and casing variations.
- TET parsing accepts numeric percentages, yes/no values, and year-score pairs.
- Appointment fields are discovered from several possible column names.
- ChangeLog list names and columns tolerate spelling/casing variants such as List name, Sheet name, HSS, Elementry, and Elementary.
- Exit and hold status are derived from text fields.

Data Security Measures

Implemented measures are limited to presentation controls and fetch safety:

- Address, pincode, and member ID display are controlled by feature flags.
- CSV/JSON fetches use cache-busting and response-type checks.
- Links opened for directions and document downloads use target="_blank" with rel="noreferrer".
- ChangeLog raw document URLs are not printed as visible table text.

Not implemented in this repository:

- Authentication.
- Authorization.
- Encryption of local records.
- Server-side audit logging.
- Role-based access control.
- Secret backend storage.
- Private Google Sheet or Drive access that is hidden from technical users.

6. Feature-by-Feature Analysis

High/Higher Secondary Priority List

Purpose: Display and rank high/higher secondary school candidates.

User value: Users can review ranked candidates, filter by department/category/pastorate/council, search, switch sort mode, and export PDFs.

Business value: Provides consistent appointment priority handling for a large candidate list.

Technical implementation:

- Mapping: mapHighSchool() in Dashboard.tsx.
- Ranking: compareHighSchoolSeniorityCandidates() and compareHighSchoolCandidates() in seniorityRules.ts.
- UI: SeniorityTable.tsx, FilterSidebar.tsx, SearchBar.tsx.
- Reports: downloadCandidatesPDF(), downloadAppointmentsReportPDF(), downloadExitRegisterPDF().

Dependencies: React, date-fns, jsPDF, jspdf-autotable.

Workflow:

1. Fetch high school rows.
2. Normalize dates, registration year, category, department, qualification, TET, pastorate, council, and appointment fields.
3. Exclude exited candidates from active list.
4. Apply filters and search.
5. Rank by seniority or appointment mode.
6. Render paginated table and optional report exports.

Elementary/Middle Priority List

Purpose: Display and rank elementary/middle school candidates.

User value: Users can review candidates by council, pastorate, category, subject, qualification, TET status, and ranking.

Business value: Standardizes elementary/middle appointment ordering and TET pass handling.

Technical implementation:

- Mapping: mapElementarySchool() in Dashboard.tsx.
- Ranking: compareElementarySchoolSeniorityCandidates() and compareElementarySchoolCandidates().
- TET pass mark: ELEMENTARY_TET_PASS_MARK = 40.

Dependencies: React, date-fns, jsPDF.

Workflow:

1. Fetch elementary rows.
2. Normalize subject/level, qualification, TET completion, registration and passing year.
3. Apply filters and search.
4. Rank by seniority or appointment mode.
5. Render table and reports.

Clergy Ordination Priority List

Purpose: Display and rank clergy ordination candidates.

User value: Users can review clergy priority by ordination year, experience, age, qualification, and home pastorate.

Business value: Provides repeatable ordination priority ordering.

Technical implementation:

- Mapping: mapClergyOrdination() in Dashboard.tsx.
- Ranking: compareClergyOrdinationCandidates().
- Filters: home pastorate and qualification.

Workflow:

1. Fetch clergy ordination rows.
2. Normalize birth date, passing year, experience, qualification, home pastorate, and appointment fields.
3. Rank by earlier passing year, more experience, and older age.
4. Render table and reports.

Search

Purpose: Search candidate records across multiple fields.

Technical implementation: searchCandidatesGeneric() in src/app/utils/helpers.ts.

Search fields include name, member ID, council, pastorate, diocese, institution, department, category, subject, level, home pastorate, experience, qualification, email, pincode, hold reason, exit type, and TET completion.

Filtering

Purpose: Narrow candidate lists using domain-specific dimensions.

Technical implementation:

- FilterSidebar.tsx.
- DropdownFilter.tsx.
- Filter groups built dynamically in Dashboard.tsx.

High school filters: department, category, pastorate, council.

Elementary filters: council, pastorate, category, subject.

Clergy filters: home pastorate, qualification.

Appointment View and Appointment Report

Purpose: Show appointed and held candidates and export appointment reports.

Technical implementation:

- Feature flag: APPOINTMENT_REPORT_ENABLED = true.
- URL search param: appointments=1.
- Appointment ranking map in buildAppointmentRankMap().
- Report export in downloadAppointmentsReportPDF().

Appointment fields include status, appointment date, institute/location, compassion reason, and hold reason.

Exit Register

Purpose: Separate exited candidates from active ranking and report them.

Technical implementation:

- Exit status detection through exitType.
- UI table in Dashboard.tsx.
- PDF export in downloadExitRegisterPDF().

ChangeLog

Purpose: Display audit/history rows for the currently selected High/Higher Secondary, Elementary/Middle, or Clergy list.

Technical implementation:

- Data source: changeLog rows from fetchGoogleSheetData() or the optional static JSON

cache.
- Mapping and filtering: mapChangeLogRows() and normalizeChangeLogSheetName() in Dashboard.tsx.
- UI: inline table beside the Exit Register controls in Dashboard.tsx.
- Documents: View opens a compact in-app preview dialog; Download uses a Google Drive download URL when a Drive file ID can be extracted.

Supported columns:

- List name or Sheet name
- Member ID
- Name
- Date
- Action
- Information Changed
- Description of Change
- Approved by
- Documents

The row can contain blank cells. HSS routes to the High/Higher Secondary list; Elementary, Elementry, Middle, and Primary route to the Elementary/Middle list; Clergy and Ordination route to the Clergy list.

Security note: Drive links are not shown as plain text and there is no Open in Drive button, but any document fetched or downloaded by the browser is still accessible to a technical user unless served through an authenticated backend proxy.

PDF Priority List Export

Purpose: Export the filtered priority list.

Technical implementation:

- Feature flag: DOWNLOAD_PDF_ENABLED = true.
- Dynamic import of pdfUtils.
- Landscape A4 PDF with logo, metadata, table, watermark, printed timestamp, and page numbering.

Visual Dashboard

Purpose: Provide chart-based summary of candidate distribution.

Technical implementation:

- DashboardVisual.tsx.
- Chart.js Doughnut charts.
- Framer Motion modal animation.

Views vary by domain:

- High school: department and category split.
- Elementary: council, pastorate, subject, and category split.
- Clergy: home pastorate and qualification.

Apply Page and Vacancy Dashboard

Purpose: Download static application forms and view school vacancies on charts and a map.

Current status: Implemented but not routed because APPLY_SECTION_ENABLED = false.

Technical implementation:

- ApplyPage.tsx.
- SchoolVacancyMap.tsx.
- Static form downloads from public/forms.
- Vacancy data from Google Sheet CSV or JSON.
- Chart.js bar/doughnut charts.

- Leaflet map with OpenStreetMap tiles.

Theme Switching

Purpose: Toggle light/dark visual mode.

Technical implementation:

- packages/ui-theme/src/theme-manager.ts.
- localStorage key: theme-mode.
- DOM class: dark.
- DOM dataset: data-theme.

Language Switching

Purpose: Toggle English and Tamil UI labels.

Technical implementation:

- src/app/i18n/language.tsx.
- localStorage key: seniority_language.
- document.documentElement.lang updated to en or ta.

7. Business Rules

High/Higher Secondary Seniority

- Earlier registration year comes first.
- If tied, earlier passing month/year comes first.
- If tied, older age comes first.
- If tied, higher TET score comes first for UG candidates.

High/Higher Secondary Appointment Mode

- PG candidates follow registration, passing, and date of birth ordering.
- UG candidates with valid TET data are prioritized over UG candidates without valid TET.
- Valid high school TET threshold is HIGH_SCHOOL_TET_PASS_MARK = 60.

Elementary/Middle Seniority

- Earlier registration year comes first.
- If tied, earlier passing year comes first.
- If tied, older age comes first.

Elementary/Middle Appointment Mode

- TET-qualified candidates are prioritized.
- Elementary TET threshold is ELEMENTARY_TET_PASS_MARK = 40.
- Tie-breaks then follow registration year, passing year, and date of birth.

Clergy Ordination

- Earlier year of passing comes first.
- If tied, higher years of experience comes first.
- If tied, older age comes first.

8. Configuration and Feature Flags

Current feature flags in src/app/config/features.ts:

- APPLY_SECTION_ENABLED = false.
- APPOINTMENT_REPORT_ENABLED = true.
- LANGUAGE_SWITCH_ENABLED = true.
- HIGH_SCHOOL_SECTION_ENABLED = true.
- MIDDLE_SCHOOL_SECTION_ENABLED = true.

- CLERGY_SECTION_ENABLED = true.
- NAVIGATION_TUTORIAL_ENABLED = true.
- HIGH_SCHOOL_TET_PASS_MARK = 60.
- ELEMENTARY_TET_PASS_MARK = 40.
- SHOW_MEMBER_ID = false.
- SHOW_ADDRESS = false.
- SHOW_PINCODE = false.
- DOWNLOAD_PDF_ENABLED = true.
- APPOINTMENT_REPORT_DOWNLOAD_ENABLED = true.

Environment variables documented or used:

- VITE_HIGH_SCHOOL_CSV_URL.
- VITE_ELEMENTARY_SCHOOL_CSV_URL.
- VITE_CLERGY_ORDINATION_CSV_URL.
- VITE_SCHOOL_VACANCY_CSV_URL.
- VITE_GOOGLE_SHEET_CSV_URL.
- VITE_HIGH_SCHOOL_DATA_URL.
- VITE_ELEMENTARY_SCHOOL_DATA_URL.
- VITE_CLERGY_ORDINATION_DATA_URL.
- VITE_GOOGLE_SHEET_DATA_URL.
- VITE_SCHOOL_VACANCY_DATA_URL.
- Sync script variables: GOOGLE_SHEET_PUB_URL, HIGH_SCHOOL_GID, ELEMENTARY_SCHOOL_GID, HIGH_SCHOOL_CSV_URL, ELEMENTARY_SCHOOL_CSV_URL, SCHOOL_VACANCY_CSV_URL.

9. Assets and Supporting Resources

Public assets:

- public/diocese-logo.png: logo used in header, splash screen, visual dashboard, and PDFs.
- public/seniority-data.json: generated/static data cache.
- public/forms/high-higher-secondary-school-form.pdf: form download.
- public/forms/elementry-middle-school-form.pdf: form download.
- public/forms/README.txt: form asset notes.

Supporting resources:

- guidelines/Guidelines.md.
- packages/ui-theme: shared theme framework.
- templates/csi-starter: reusable CSI starter application.

10. Deployment and Runtime

Static Hosting

The app builds to dist and can be deployed on a static host. vercel.json rewrites all paths to index.html, enabling browser-router deep links.

Docker Hosting

The Dockerfile:

1. Uses node:20-alpine to install dependencies and run npm run build.
2. Uses nginx:1.27-alpine to serve dist.
3. Exposes port 80.

Local Tooling Observation

In the current shell session, node and npm were not available on PATH, so a local npm run build verification could not be performed here. The repository does contain node_modules, package-lock.json, and a Node-oriented Docker build path.

11. Risks, Gaps, and Recommendations

Observed Gaps

- README.md documents npm run sync:sheet and npm run sync:sheet:watch, but those scripts are not present in package.json.
- The Apply route is implemented but disabled by current feature flag.
- ATTRIBUTIONS.md exists but is empty.
- Static form PDFs in public/forms are only 431 bytes each in this workspace, which should be reviewed to ensure they are valid production forms.
- No automated tests are present in the discovered file set.
- No authentication or authorization layer exists in this repository.

Recommendations

- Add sync:sheet and sync:sheet:watch scripts or update README.md.
- Add unit tests for ranking rules, CSV parsing, date parsing, and appointment handling.
- Add integration tests for data loading failures and HTML response detection.
- Validate bundled PDF forms.
- Decide whether member ID, address, and pincode are safe to ship in source data if hidden by UI flags.
- Add dependency/license attribution if required by the organization.
- Consider row virtualization if candidate counts grow substantially beyond current volumes.

12. Source Inventory

Key application files reviewed:

- README.md.
- package.json.
- vite.config.ts.
- vercel.json.
- Dockerfile.
- sync-google-sheet-to-json.js.
- src/app/App.tsx.
- src/app/routes.tsx.
- src/app/pages/Root.tsx.
- src/app/pages/Dashboard.tsx.
- src/app/pages/ApplyPage.tsx.
- src/app/config/features.ts.
- src/app/config/seniorityRules.ts.
- src/app/utils/fetchGoogleSheetData.ts.
- src/app/utils/helpers.ts.
- src/app/utils/pdfUtils.ts.
- src/app/components/Header.tsx.
- src/app/components/FilterSidebar.tsx.
- src/app/components/SeniorityTable.tsx.
- src/app/components/DashboardVisual.tsx.
- src/app/components/SchoolVacancyMap.tsx.
- src/app/components/AppGuidelines.tsx.
- src/app/i18n/language.tsx.
- packages/ui-theme/src/theme-manager.ts.
- packages/ui-theme/README.md.
- templates/csi-starter/README.md.
- public/seniority-data.json.
- data/high-higher-secondary-converted.csv.
- elementary-mock-data.csv.